By the end of Module 2, you'll be able to move from "I asked AI to build something" to "I designed a system and used AI to help me think through it."

📕 When We'll Meet

- 🗓️ Tuesday, October 20th at 4:20PM EDT
- 📊 Link to Slides

Tinker: ByteBites

ℹ️ Activity Overview

⏰ ~45 minutes

You have joined the development team of ByteBites, a new campus food ordering app that wants to feel fast, personalized, and a little bit smart. The problem is that the current prototype is a mess. The menu structure is inconsistent, orders are unreliable, and the so-called AI recommendations suggest soup as a dessert pairing. No one knows how the system is supposed to fit together.

Your task is to design and build the core architecture of the app using Python classes and simple algorithms. You'll use AI as a design partner, but you must verify and refine everything it produces. By the end, you'll have created a clean and functional version of the system that makes sense to a human engineer.

You'll practice reading vague design requests, turning them into clear system components, generating UML-style diagrams with AI, translating those diagrams into code, and testing your work with small examples. You'll also learn how to guide an AI tool into producing helpful scaffolds rather than over-engineered solutions.

🎯 Goals

By completing this activity, you will be able to...

- Use object-oriented design to break a feature request into core components.
- Represent and manipulate structured data using simple Python classes and containers.
- Apply algorithmic reasoning to tasks such as sorting, filtering, and validation.
- Use UML-style diagrams to think about system structure before writing code.
- Incorporate AI-produced code and diagrams as inputs to your workflow.

✏️ Tinker Instructions

▼ Part 1: System Design (UML and AI Collaboration)

⏰ **~10 mins**

You have received a raw feature request from the ByteBites product manager. Your task is to translate that request into a clear system architecture: create a workspace, prepare your AI tooling, identify the four core components (classes), and use AI (with project context) to draft a UML-style class diagram. You'll evaluate and refine the diagram until it matches exactly what the request describes.

1a. Set Up Your Working Environment

Before you begin designing, set up your working environment.

- Create a new folder on your computer named `bytebites_tinker_activity`.
- Download the provided reference file into that folder. You'll edit and use it later!
- Open VS Code and choose **File → Open Folder…**, then select the new folder.
- Open the terminal and initialize a Git repository:

```
git init
```

- Create your first commit so VS Code recognizes this as an active Git workspace:

```
git add .
git commit -m "initial commit"
```

1b. Create the Spec File

- In the project root, create a Markdown file named `bytebites_spec.md`.
- Copy the **Client Feature Request** (below) into the file.
- Add a heading "Candidate Classes" and list the four nouns you believe the system must model. Use your own judgment here; the goal is to identify the core data objects the system needs.

Client Feature Request

We need to build the backend logic for the ByteBites app. The system needs to manage our customers, tracking their names and their past purchase history so the system can verify they are real users.

These customers need to browse specific food items (like a "Spicy

Burger" or "Large Soda"), so we must track the name, price, category, and popularity rating for every item we sell.

We also need a way to manage the full collection of items — a digital list that holds all items and lets us filter by category such as "Drinks" or "Desserts".

Finally, when a user picks items, we need to group them into a single transaction. This transaction object should store the selected items and compute the total cost.

1c. Draft a UML Diagram Using Your AI Coding Assistant

- Open your AI coding assistant's chat in VS Code.
- Attach or reference `bytebites_spec.md` as context.
- Choose a chat mode (Ask, Edit, or Plan) and explore how it interprets your spec.
- Craft a prompt that asks your AI coding assistant to generate a UML-style class diagram for your four classes. Think about what information the model needs and how to phrase your request clearly.

  If you're unsure how to start, focus on *describing the system*, not on formatting or syntax. You can explain what the system does and what classes you identified, then ask your AI coding assistant to visualize or organize that information. If the output is confusing, ask follow-up questions or request a simpler version.
- Review the diagram you receive.
- Does it add classes not in your candidate list?
- Does it miss important attributes?
- Does anything look inconsistent with the feature request?

A useful diagram should help you understand the system. Ask yourself:

- Can you explain what each class represents in one sentence?
- Do the attributes match what the feature request describes?
- If a class or relationship surprises you, that is a signal to question it.
- Save the output in a temporary file such as `draft_from_copilot.md`.

1d. Add a reference file for ByteBites

- Open `ByteBites_Design_Reference.md` and fill in the `Behavioral Instructions` section with specific guidance you want to consistently apply. Think about the scope of work and other wide-ranging decisions.
- In VS Code, start a new chat session or Claude Code conversation.
- **(Option 1) Copilot users:** Add the file to your conversation by clicking + at the bottom of the window, choosing files and folders, then typing the file name `ByteBites_Design_Reference.md`. Click it in the list or hit enter when it's highlighted to select it.
- **(Option 2) Claude:** Add the file to your conversation by clicking + at the bottom of the window. If you downloaded the file into your existing workspace, choose **Add context**. If you downloaded it somewhere else, choose **Upload from computer**. Either way, select the `ByteBites_Design_Reference.md` file from the list.

  The reference file is a way to give your AI coding assistant extra context so it behaves more predictably. Instead of repeating the

same information in every chat message, you encode it once. Think of it as setting "ground rules" for how AI suggestions should be shaped in this project.

1e. Use the Reference file to Draft and Refine Your UML Diagram

- In the chat window you opened in the previous step (with the reference file attached), ask the assistant to produce a revised UML diagram for your four classes.
- Compare the new diagram to the earlier draft. Notice how the behavior differs from the less-bounded version.
- Iterate with follow-up messages of your design until the diagram matches the feature request accurately.
- Save the final version as `bytebites_design.md`.
- Commit it:

```
git add bytebites_design.md
git commit -m "Add final UML design diagram"
```

📍**Checkpoint**: You now have two diagrams to compare: one from standard chat and one with extra guidance. Reflect on how the outputs differed and why the chat with a reference produced a design closer to the specification. Your final saved diagram should reflect your verified interpretation of the ByteBites feature request.

▼ Part 2: Code Implementation

⏰ **~10 mins**

In this section you'll turn your UML diagram into Python class scaffolds. You'll use your AI coding assistant's chat to plan and draft your code, then review and refine it yourself with files

referenced as needed. Your goal is to practice a full AI-assisted workflow where you define intent, generate structured code, and evaluate the output carefully.

2a. Create the Implementation File

- In your project folder, create a file named `models.py` in your project.
- Add brief comments at the top summarizing the four classes you designed.
- Commit the file.

```
git add models.py
git commit -m "Add empty models.py with class list"
```

2b. Use Your AI Coding Assistant to Plan Your Class Structure

- Open your AI coding assistant's chat and switch to **Plan Mode**.

  Plan mode works best when you describe intent, not syntax. Focus on what each class represents and what information it should store. If the plan feels too detailed or introduces features you didn't design, you can ignore or revise it. The plan is a suggestion, not a requirement.
- Attach your spec file and UML diagram.
- Ask your AI coding assistant for a high-level plan for implementing your four classes.
- Review the plan and adjust it so that it matches your diagram and the ByteBites spec.

2c. Draft Class Scaffolds with Your AI Coding Assistant

- Begin adding empty class definitions in `models.py` (for example: `class Customer:`).
- For each class, decide what context to give your AI coding assistant, then use **ask**, **edit**, or **plan** mode to request a draft scaffold.

  At this stage, a scaffold is just the basic shape of a class. It usually includes the class name, a constructor, and attributes that store data. You're not expected to implement real logic yet. If your class feels "empty but structured," you're on the right track.
- Examine the generated code carefully.
- Remove anything that does not appear in your UML diagram.
- Add anything your AI coding assistant missed.
- Make sure the structure reflects your own design choices.

2d. Refine By Adding the Reference File

- When your scaffolds are in place, add your reference file (or any others you think would be useful) to your AI coding assistant's chat.
- Ask it to review one class at a time and propose refinements that keep the implementation aligned with the ByteBites spec and all available information.

2e. Run a Quick Manual Check

- Create a few sample objects in a temporary script.
- Print or inspect them to confirm that attributes are sorted correctly. Make any adjustments you discover during this testing.

  You're not writing formal tests yet. You're just creating a few

objects and checking that Python runs without errors and stores the data you expect. If you can create objects and print their attributes successfully, your scaffolds are working.

- Save and commit your completed scaffolds:

```
git add models.py
git commit -m "Add initial scaffolds for
ByteBites classes"
```

📍**Checkpoint**: You now have four class scaffolds that reflect your UML diagram. You used your AI coding assistant's chat to plan and draft the structure, refined the results, and confirmed that your classes behave as expected.

▼ Part 3: Algorithmic Enhancements

⏰ **~10 mins**

In this section you'll implement the core algorithms that make your ByteBites system functional. You'll use your AI coding assistant's chat to help plan and draft the logic, then evaluate and refine the results yourself. You can add references to help with method review once you have working drafts. The goal is to practice using AI as a collaborator while keeping full responsibility for correctness.

3a. Plan Your Algorithmic Work

- Open your AI coding assistant's chat and switch to **Plan Mode**.

  Plan mode is meant to help you think ahead before writing code. A good plan names the methods you need and describes what each one should do in plain language. If the plan includes code or details you do not recognize, treat it as optional guidance, not a requirement.

- Attach your spec file and UML diagram.
- Ask your AI coding assistant to outline the methods your system needs for filtering, sorting, and computing totals.

  If you feel unsure what methods to implement, revisit your UML diagram and spec file. The plan should reflect decisions you already made, not invent new responsibilities for your classes.
- Review and adjust the plan so that it matches the design you created. Treat this plan as a guide rather than a set of instructions.

  3b. Draft Each Method with Your AI Coding Assistant
- For each method (filtering, sorting, total calculation), start by writing a simple method signature and a brief comment about the intended behavior.
- Highlight your method or the surrounding context, then use **ask**, **edit**, or **plan** mode in your AI coding assistant's chat to request a first draft of the implementation.

  Ask mode is useful when you want ideas or a first draft from scratch. Edit or plan mode is useful when you already have a method stub or partial logic and want help improving or completing it. If one mode gives confusing output, try the other.
- Review the generated code carefully.
- Does it follow your UML design?
- Does it use the attributes your classes actually have?
- Does it avoid adding unnecessary features?
- Modify the code until it behaves correctly.

  3c. Refine Methods with Additional Reference Files

- When your method works, highlight the method and add any additional references you think may help with context, style, or structure.
- Ask the assistant to review the method for clarity, correctness, or structure. Use your judgment to decide whether to adopt the suggestions. Only keep refinements that improve the method without introducing inconsistencies.

3d. Test Your Methods Manually

- At the bottom of `models.py` or in a temporary script, create a small scenario that exercises your new methods. Include adding items, sorting menus, filtering categories, and computing an order total.
- Run the code to verify that each method behaves as intended.
- Make corrections as needed.
- Save and commit your updated code:

```
git add models.py
git commit -m "Implement algorithmic methods with
AI-assisted workflow"
```

📍**Checkpoint**: You planned your work with your AI coding assistant's chat, drafted each method using ask or edit mode, verified correctness yourself, and used additional references for refinement. Your ByteBites system now has fully functioning filtering, sorting, and total calculation logic created through a structured AI-assisted workflow.

▼ Part 4: Testing and Validation

⏰ **10 mins**

In this section you'll write tests by first describing what the test should do in comments, then using one of your AI coding assistant's modes to turn those comments into `pytest` code. After you review and correct the draft, you'll use additional references to refine it further so the final test matches the system specification.

4a. Set Up Your Test File

- Create a new file named `test_bytebites.py` in your project.
- Import the classes you need from `models.py`.
- Commit the empty file:

```
git add test_bytebites.py
git commit -m "Create test file"
```

4b. Use Your AI Coding Assistant to Explore Test Ideas

- Open your AI coding assistant's chat and switch to a mode you think will help you start (Ask, Edit, or Plan).
- Describe the behaviors you want to test, such as category filtering, sorting, and total calculation.
- Use your AI coding assistant's suggestions to inspire or structure your test functions. You may choose to generate a full test draft or simply gather ideas.

  When thinking about tests, describe what the system should do from the outside. Avoid testing internal details like how a loop is written. If you can explain the behavior in one sentence, it is usually a good test candidate.

  A robust test suite doesn't just check if the code works; it checks where it might break.

- **Logic Tests (The "Happy Path"):** Focus on the core functionality.

If a user adds a $10 burger and a $5 soda, does the total equal $15?

- *Template:* `test_calculate_total_with_multiple_items`
- **Edge Cases (The "What If?"):** Focus on boundaries and empty states. What happens if a user opens a transaction but adds nothing? Does the system return $0 or crash?
- *Template:* `test_order_total_is_zero_when_empty`

Quick Checklist for a "Good" Test:

- **One Sentence:** If you can't describe the test's goal in one sentence (e.g., "Verify that filtering 'Drinks' only returns liquid items"), it's too complex.
- **Isolation:** Avoid testing how the code is written (loops/if-statements); focus on the output it produces.

4c. Draft Your Tests

- Write test functions in `test_bytebites.py` that cover the key behaviors of your system. Include at least:
- a test for order totals,
- a test for empty totals, and
- a test for filtering menu items by category.
- If you want help drafting or refining a specific test, highlight it and use your AI coding assistant's chat in ask or edit mode.

  It is okay if you are unfamiliar with `pytest`. Many engineers use AI tools to draft test structure and syntax, then learn by reading and modifying the result. Focus on whether the test checks the right behavior.

4d. Refine Tests with Additional References

- When you have test drafts that run, highlight one test at a time and add more reference files to your context.
- Ask the assistant to review the test for clarity, correctness, or consistency with the specification.
- Apply only the refinements that improve the test and match your own understanding of the system.

4e. Run and Debug Your Test Suite

- Use `python -m pytest` in the terminal to run your tests.
- Review any failures and decide whether the test or your implementation needs adjustment. Use your AI coding assistant's chat if you want help diagnosing issues, but make final decisions yourself.

  When a test fails, read the error message from top to bottom. Look for:

- which test failed,
- what value was expected,
- what value was actually returned.

  You can paste the error message into your AI coding assistant's chat and ask for help interpreting it, but you should decide what to change.

- Continue refining until all tests pass.
- Commit your final test suites:

```
git add test_bytebites.py
git commit -m "Add validated tests for ByteBites"
```

📍**Checkpoint**: You now have a working test suite created through a mix of your own reasoning and AI-assisted drafting. You explored several AI coding assistant chat modes, refined individual tests, and verified correctness through automated execution. Your ByteBites system is fully implemented and tested.

▼ After Class:

🏁 **Solution repo:** ai110-module2tinker-bytebites-solution

This is a finished ByteBites workspace — the spec, the verified UML design, the four classes in `models.py`, and a passing `pytest` suite — one complete, working example of what this Tinker is aiming at, not the only correct answer. Try the activity on your own first (the solution is most useful *after* your own attempt), then use it to check whether your design and code met the objectives.

📫 Reminder: Project 1 is due by **Friday, October 23rd at 11:59PM EDT**.

Show What You Know: Game Glitch Investigator

ℹ️ Project Overview

⏰ ~3 hours

Your AI pair programmer decided to be helpful and built you a Python game. Only problem? It's full of glitches. Sometimes the game never ends, the hints lie, or the score goes haywire. Your mission: become the **Game Glitch Investigator** -- a developer who can diagnose, explain, and repair AI-generated code responsibly. You'll use your AI coding assistant to find bugs, plan refactors across multiple files, and generate automated tests.